Análise de arquivo suspeito

Share

exec.ko

KO

Analise o arquivo em anexo passo a passo e determine ao final: este arquivo é benigno ou é um malware?

Consultando a skill de leitura de arquivos

Vou começar consultando a skill de leitura de arquivos e depois fazer a análise estática do módulo.

Ran 6 commands, read a file

Fiz uma análise estática, sem executar o arquivo. Estes foram os passos:

1. Identificação do tipo. `exec.ko` é um ELF 64-bit relocável para x86-64, ou seja, um **módulo de kernel Linux**. Tem 684.776 bytes, mas só cerca de 3,6 KB são código executável; o resto é debug/BTF. SHA-256: `855f46e10ec521fb0e3404ea86f65130830321ea85754019af5b7e3fc04f9d52`.

2. Metadados declarados (`.modinfo`). O módulo traz `description=LKM rootkit`, `name=diamorphine` e `author=m0nad`, compilado para o kernel `7.1.13+parrot7-amd64`. O nome do arquivo (`exec.ko`) não bate com o nome interno, e nome de arquivo nunca é critério confiável.

3. Símbolos e strings. Há funções como `hacked_kill`, `hacked_getdents`, `hacked_getdents64`, `give_root`, `module_hide`, `module_show` e `get_syscall_table_bf`. Há também strings como `sys_call_table`, `kallsyms_lookup_name` e `diamorphine_secret`, que é um prefixo-marcador para esconder arquivos.

4. Verificação do código real, não só dos nomes. Cruzei as relocações de cada função com as APIs do kernel que ela de fato chama:

Função	APIs do kernel usadas	Comportamento
`hacked_getdents` / `hacked_getdents64`	`copy_from_user`, `copy_to_user`, `memmove`, `init_task`	Interceptam a listagem de diretórios e removem entradas, ocultando arquivos e processos
`hacked_kill`	`prepare_creds`, `commit_creds`, `list_add/del`	Usa o syscall `kill` como backdoor, acionado por sinais especiais
`give_root`	`prepare_creds`, `commit_creds`	Concede root a um processo
`module_hide` / `module_show`	`list_del` / `list_add` em `__this_module`	Tira o módulo da lista do kernel (some do `lsmod`); é chamado já no carregamento
`resolve_sym` / `get_syscall_table_bf`	`register_kprobe`, `kallsyms_lookup_name`	Localiza a tabela de syscalls para instalar os hooks

O código faz o que os nomes anunciam.

5. O que o módulo não tem. Não há imports de rede, arquivos, execução de processos ou criptografia, nem URLs, IPs ou comandos de shell nas strings. Não há C2, exfiltração ou persistência embutida: é um rootkit local de ocultação, com backdoor de escalonamento de privilégio.

6. Proveniência. O debug info mostra GCC 14.2.0, fonte `diamorphine.c` e diretório de build `/home/user/Documents/malwares_rep/Diamorphine`, num Parrot OS. Isso indica compilação local do código público do Diamorphine, provavelmente em ambiente de estudo ou laboratório.

Veredicto: é malware. É um build do rootkit de kernel open-source **Diamorphine**, com confiança alta, porque metadados, símbolos e comportamento do código convergem. Nenhuma função legítima justifica um módulo que se esconde do sistema, oculta processos e arquivos e concede root por sinal.

Se foi você quem compilou isso em um laboratório, não há infecção, mas continua sendo malware funcional e não um falso positivo.

Recomendações

- Não carregue com `insmod` fora de uma VM isolada com snapshot. Ele foi compilado para um kernel específico e em outros normalmente seria recusado, mas isso não é garantia.
- Se encontrou o arquivo em um sistema inesperado, isole a máquina e não confie nas ferramentas dela, pois o rootkit esconde coisas delas. Investigue por mídia externa ou dump de memória e reinstale a partir de mídia confiável.
- Para corroborar, consulte o SHA-256 no VirusTotal. Sem rede aqui, não pude fazer isso.

- Como prevenção, use Secure Boot com assinatura de módulos obrigatória, kernel lockdown e `kernel.modules_disabled=1` após o boot em servidores.

A análise foi estática e não li o desassembly linha a linha, mas a convergência das evidências torna a conclusão sólida. Se quiser, preparo um checklist de detecção e resposta a incidente para rootkits desse tipo.

Qual é a categoria principal deste malware?

Definindo a subcategoria de rootkit para Linux

Rootkit, mais especificamente um **rootkit de kernel do tipo LKM** (Loadable Kernel Module) para Linux. Na matriz MITRE ATT&CK, isso corresponde à técnica T1014 (Rootkit).

O que define essa categoria é que o objetivo principal do módulo é **ocultação e controle persistente dentro do kernel**: ele se esconde da lista de módulos, oculta arquivos e processos interceptando syscalls e mantém um acesso privilegiado escondido. Como característica secundária, funciona como **backdoor local de escalonamento de privilégio**, já que concede root por sinais especiais enviados via `kill`.

Ele não se enquadra em outras categorias comuns como ransomware, trojan/RAT, worm, stealer ou botnet. Não há código de rede, criptografia, propagação ou coleta de dados. Por isso, normalmente é usado como **componente de pós-exploração**, ou seja, depois que o atacante já tem acesso root à máquina, para esconder sua presença.